



Home

# Laborator: tipuri algebrice, funcții recursive, inducție, demonstrații

- Back to [PLP - Fall 2024](#)

## Setup

Se vor considera cunoscute următoarele definiții:

```
Inductive Nat := 0 : Nat | S : Nat -> Nat.
```

```
Definition one := S 0.
```

```
Definition two := S one.
```

```
Definition three := S two.
```

```
Definition four := S three.
```

```
Definition five := S four.
```

## Exerciții de antrenament (testarea unor funcții):

- Se dă următoarea funcție de egalitate a două elemente de tip Nat:

```
Fixpoint eq_Nat (n m : Nat) :=
  match n, m with
  | 0, 0      => true
  | 0, S _    => false
  | S _, 0    => false
  | S n', S m' => eq_Nat n' m'
  end.
```

Testați funcția eq\_Nat rulând:

```
Compute eq_Nat 0 0.
```

```
Compute eq_Nat one 0.
```

```
Compute eq_Nat one one.
```

```
Compute eq_Nat one four.
```

```
Compute eq_Nat five four.
```

```
Compute eq_Nat five five.
```

- Se dă funcția de adunare peste Nat:

```
Fixpoint add (m n : Nat) : Nat :=
  match m with
  | 0 => n
  | S m' => S (add m' n)
  end.
```

**Testați funcția add rulând:**

Compute add one one.  
 Compute add one five.  
 Compute add five one.  
 Compute add five 0.  
 Compute add 0 five.

- Se dă o funcție `max` care returnează maximumul dintre două elemente de tip `Nat`:

```
Fixpoint max (m n : Nat) : Nat :=
  match m with
  | 0 => n
  | S m' => match n with
            | 0 => m
            | S n' => S (max m' n')
          end
  end.
```

**Testați funcția max rulând:**

Compute max one two.  
 Compute max 0 one.  
 Compute max one 0.  
 Compute max four five.  
 Compute max five four.

**Exerciții propuse:**

1. (1 punct) Scrieți o funcție `le_Nat` care primește doi parametri `m` și `n` de tip `Nat` și returnează `true` dacă `m` este mai mic sau egal decât `n`.

```
Fixpoint le_Nat (m n : Nat) : bool :=
  ...
```

Testați funcția `le_Nat` pe următoarele exemple (punctajul se acordă în cazul în care toate testele returnează corect rezultatul din comentarii):

```
Compute le_Nat 0 0. (* true *)
Compute le_Nat 0 one. (* true *)
Compute le_Nat one 0. (* false *)
Compute le_Nat one one. (* true *)
Compute le_Nat one five. (* true *)
Compute le_Nat five one. (* false *)
Compute le_Nat five four. (* false *)
Compute le_Nat five five. (* true *)
```

2. (1 punct) Demonstrați următoarea leamnă (hint: puteți utiliza `induction` și `inversion`):

```
Lemma le_then_0 :
  forall n,
    le_Nat n 0 = true ->
      n = 0.
Proof.
  ...
Qed.
```

3. (1 punct) Demonstrați următoarele leme prin inducție:

```
Lemma le_refl:
  forall x,
    le_Nat x x = true.
Proof.
  ...
Qed.
Lemma le_Trans :
  forall m n p,
    le_Nat m n = true ->
    le_Nat n p = true ->
    le_Nat m p = true.
Proof.
  ...
Qed.
```

4. (1 punct) Formulați și demonstrați prin inducție în Coq proprietatea:  $x$  este mai mic sau egal decât  $x + y$ , pentru orice  $x$  și  $y$ .
5. (1 punct) Dacă  $x$  este mai mic sau egal decât  $y$ , atunci  $x$  este mai mic decât  $y + z$ , pentru orice numere naturale  $x, y$  și  $z$ . Formulați această proprietate ca leamnă în Coq și demonstrați-o prin inducție.
6. (1 punct) Dacă  $x$  este mai mic sau egal decât  $y$ , atunci maximul dintre  $x$  și  $y$  este mereu  $y$ , pentru orice numere naturale  $x$  și  $y$ . Formulați această proprietate ca leamnă în Coq și demonstrați-o prin inducție.
7. (1 punct) Dacă  $x$  nu este mai mic sau egal decât  $y$ , atunci maximul dintre  $x$  și  $y$  este mereu  $x$ , pentru orice numere naturale  $x$  și  $y$ . Formulați această proprietate ca leamnă în Coq și demonstrați-o prin inducție.

Coq Reference Manual: <https://coq.inria.fr/distrib/current/refman/>

Last updated: 01/09/2025 09:05:10 - Copyright © Andrei Arusoaie

Facultatea de Informatică, Universitatea „Alexandru Ioan Cuza” din Iași